SISTEMES DE CONTROL DE VERSIONS

INTRODUCCIÓ **3**

TERMINOLOGIA **4**

TIPUS DE SCV **5**

3.1 Local 5

3.2 Centralitzat 6

3.3 Distribuït 7

GIT **9**

4.1 Fonaments de Git 9

4.2 Instal·lació 11

4.3 Configuració inicial 12

4.4 Obtenció d'un repositori 12

4.5 Ordres bàsiques 14

4.6 Ordres avançades 19

4.7 Treballar amb branques 23

4.8 Etiquetatge 28

GITFLOW **31**

5.1 Per què implementar-lo? 32

5.2 Quan es recomana implementar-lo? 32

5.3 Funcionament 32

5.4 Extensió flow de Git (Git-Flow) 34

6. GITHUB PAGES **37**

7. BONES PRÀCTIQUES EN GIT **38**

8. BIBLIOGRAFIA **39**

SISTEMES DE CONTROL DE VERSIONS

Un control de versions és un sistema que registra els canvis realitzats en un fitxer o conjunt de fitxers al llarg del temps, de manera que pugues recuperar versions específiques més endavant. Aquest tipus de control no sols s'aplica en el món del desenvolupament; també el podem trobar present en accions tan quotidianes com la gestió de documents físics (sobre els quals podem crear diferents versions establint-los noms distints, per exemple), l'edició de documents col·laboratius (Google Docs) o els wikis.

> En enginyeria del programari el control de versions el podem definir com un sistema que registra els canvis realitzats sobre un fitxer o conjunt de fitxers al llarg del temps de tal manera que siga possible recuperar versions específiques més endavant. Aquest control es pot dur a terme utilitzant eines com CVS, Mercurial, Git, etc.

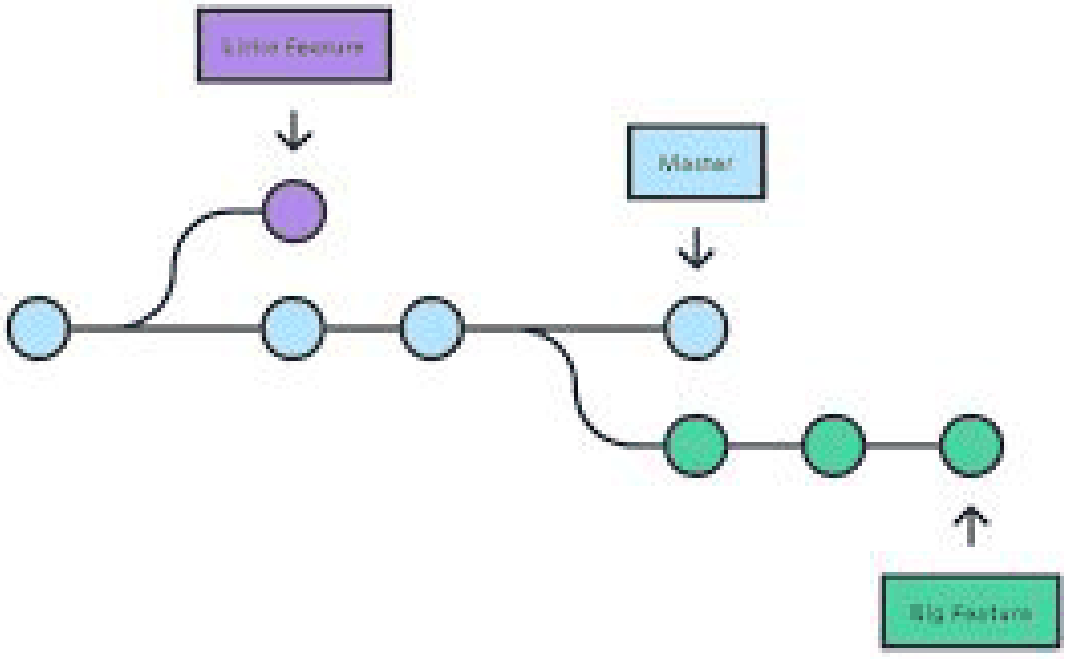
Encara que la gestió dels sistemes de control de versions se sol fer per línia d'ordres, les versions més modernes d'aquest tipus d'eines solen oferir la possibilitat de fer la gestió a través d'interfícies gràfiques fàcils d'usar. A més, moltes d'elles s'integren amb els IDE més populars.

1. INTRODUCCIÓ

En projectes de desenvolupament de programari d'una determinada envergadura resulta imprescindible l'ús d'un SCV, ja que ens ofereix la possibilitat de desfer canvis no desitjats i, fins i tot, tornar a versions anteriors del nostre desenvolupament.

L'estructura d'un sistema de control de versions **es pot interpretar com un arbre**, en què el **tronc és el lloc on es pugen els canvis realitzats sobre l'aplicació** i sobre el qual poden crear-se **branques per poder treballar sobre noves característiques o solucionar errors** que més tard s'afegiran al tronc si fóra necessari.

No hi ha un límit en el nombre de branques que podem crear sobre el tronc principal del nostre SCV. Per exemple, en la imatge següent disposem d'una branca principal i dues branques utilitzades per implementar la Little Feature i la Big Feature respectivament.



2. TERMINOLOGIA

Els distints sistemes de control de versions solen utilitzar una terminologia molt semblant que és interessant conéixer per poder treballar correctament. Entre els conceptes més destacats trobem:

- **Repositori** (repository): fa referència al lloc on es guarden els fitxers actuals i l'històric de canvis.
- **Tronc** (trunk o master): és la branca principal d'un repositori de la qual ixen la resta de branques.
- **Branca** (branch): és una bifurcació del tronc o branca mestra de l'aplicació que conté una versió independent de l'aplicació i en la qual poden aplicar-se canvis sense que afecten ni el tronc ni altres branques.
- **Capçalera** (head o tip): fa referència a la versió més recent d'una determinada branca o del tronc. El tronc i cada branca tenen la seua pròpia capçalera, però per referir-se a la del tronc de vegades s'utilitza el terme HEAD, en majúscules.
- **Còpia de treball** (working copy): fa referència a la còpia local dels fitxers que s'han copiat del repositori, que és sobre la qual es fan els canvis (és a dir, s'hi treballa, d'ací el nom) abans d'afegir aquests canvis al repositori.
- **Bifurcació** (fork): consisteix a crear un nou repositori a partir d'un altre. Aquest nou repositori, al contrari que en el cas de la clonació, no està lligat al repositori original i es tracta com un repositori diferent.
- **Clonar** (clone): consisteix a crear un nou repositori que és una còpia idèntica d'un altre, ja que conté les mateixes revisions.
- **Pujar** (commit o check in): és afegir els canvis locals al repositori. Cal destacar que no els envia al servidor; els canvis queden emmagatzemats en el repositori local, que s'ha de sincronitzar.

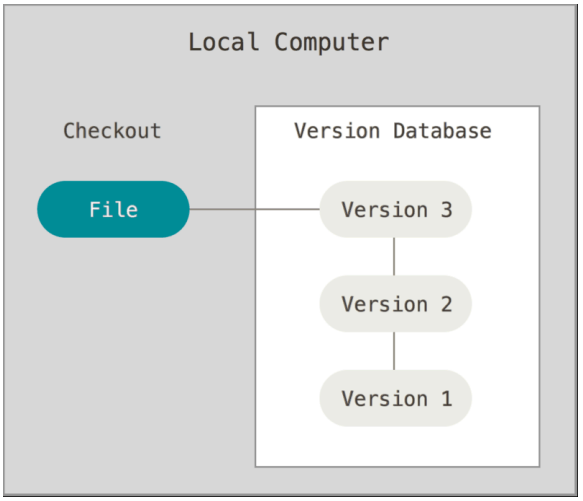
- **Baixar** (check out): és copiar en l'àrea de treball local una versió des d'un repositori local, un repositori remot o una branca diferent.
- **Pull**: és l'acció que copia els canvis d'un repositori (habitualment remot) en el dipòsit local. Aquesta acció pot provocar conflictes.
- **Push o fetch**: són accions utilitzades per afegir els canvis del repositori local a un altre repositori (habitualment remot). Aquesta acció pot provocar conflictes.
- **Canvi** (change o diff): representa una modificació concreta d'un document sota el control de versions.
- **Sincronització** (update o sync): és l'acció de combinar els canvis fets al repositori amb la còpia de treball local.
- **Conflicte** (conflict): es produeix quan s'intenten afegir canvis a un fitxer que ha sigut modificat prèviament per un altre usuari. Abans de poder combinar els canvis amb el repositori caldrà resoldre el conflicte.
- **Bloqueig** (lock): alguns sistemes de control de versions, en lloc d'utilitzar el sistema de fusions, el que fan és bloquejar els fitxers en ús, per la qual cosa només pot haver-hi un únic usuari modificant un fitxer en un moment donat.
- **Fusionar** (merge o integration): és l'acció que es produeix quan es volen combinar els canvis d'un repositori local amb un de remot i es detecten canvis en el mateix fitxer en tots dos repositoris i es produeix un conflicte. Per resoldre aquest conflicte s'han de fusionar els canvis abans de poder actualitzar els repositoris. Aquesta fusió pot consistir a descartar els canvis d'un dels dos repositoris o editar el codi per incloure els canvis del fitxer en tots dos costats. Cal destacar que és possible que un mateix fitxer presente canvis en molts punts diferents que han de ser resolts per poder donar la fusió per finalitzada.
- **Versió** (version o revision): és el conjunt de canvis en un moment concret del temps. Es crea una versió cada vegada que s'afigen canvis a un repositori.
- **Etiqueta** (tag, label o baseline): permet afegir una etiqueta a una pujada per poder identificar aquesta pujada concreta d'una manera més comprensible. Per exemple, es pot etiquetar la primera versió d'un programari (1.0) o una versió en la qual s'ha solucionat un error important.
- **Tornar a la versió anterior** (revert): descarta tots els canvis produïts en la còpia de treball des de l'última pujada al dipòsit local.

3. TIPUS DE SCV

Podem diferenciar 3 tipus de sistemes de control de versions: local, centralitzat i distribuït.

3.1 Local

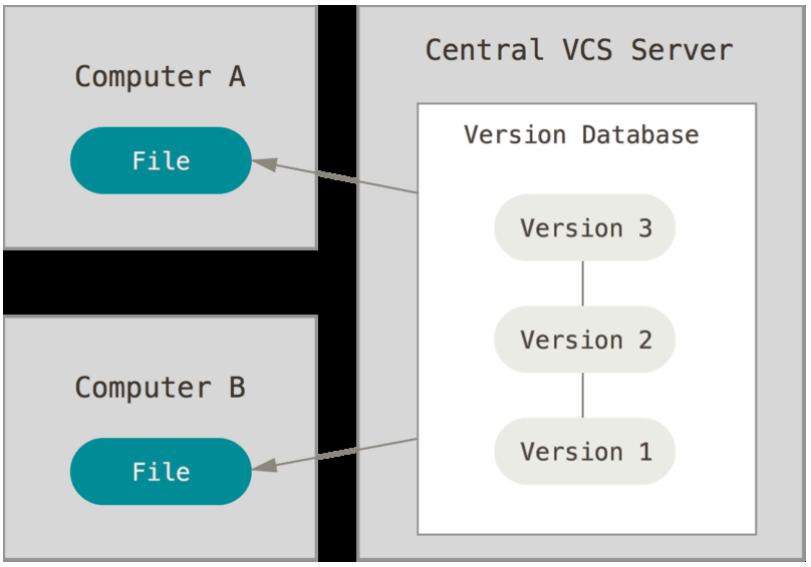
El més senzill de tots. Consisteix a copiar els fitxers a un altre directori amb la finalitat de mantindre un historial de canvis en local (ja siga còpies o mitjançant registres en una base de dades). Aquest mètode és molt **comú però també és tremendament propens a errors**. És fàcil oblidar en quin directori et trobes i guardar accidentalment en el fitxer equivocat o sobreescriure fitxers que no volies.



Una de les eines de control de versions més populars va ser un sistema anomenat RCS, que encara podem trobar en sistemes com Mac OS X. Aquesta eina funciona guardant conjunts de pedaços en un format especial en disc, i és capaç de recrear com era un fitxer en qualsevol moment a partir d'aquests pedaços.

3.2 Centralitzat

Els CVS locals, encara que senzills d'utilitzar, porten amb ells una problemàtica molt comuna: no són vàlids quan el que es necessita és dur a terme un projecte que implique col·laboració entre diverses persones. Els sistemes de control de versions centralitzats (CVCS per les seues sigles en anglés) van ser desenvolupats per solucionar aquest problema. Aquests sistemes **tenen un únic servidor que conté tots els fitxers versionats i diversos clients que descarreguen els fitxers des d'aquest lloc central**. Aquest **ha sigut l'estàndard** per al control de versions durant **molts anys**.



Com podem deduir de l'anterior, els principals **avantatges** que presenten els CVS centralitzats són:

- Totes les persones saben fins a cert punt en què estan treballant els altres col·laboradors del projecte.

- **És més fàcil administrar** un CVCS que haver de bregar amb bases de dades locals en cada client.

Com a **inconvenients**, podem remarcar el fet que només hi ha **un punt únic de fallada** que representa el servidor centralitzat. Quan tens tota la història del projecte en un mateix lloc, t'arrisques a perdre-ho tot (cosa que també ocorre en els SCV locals).

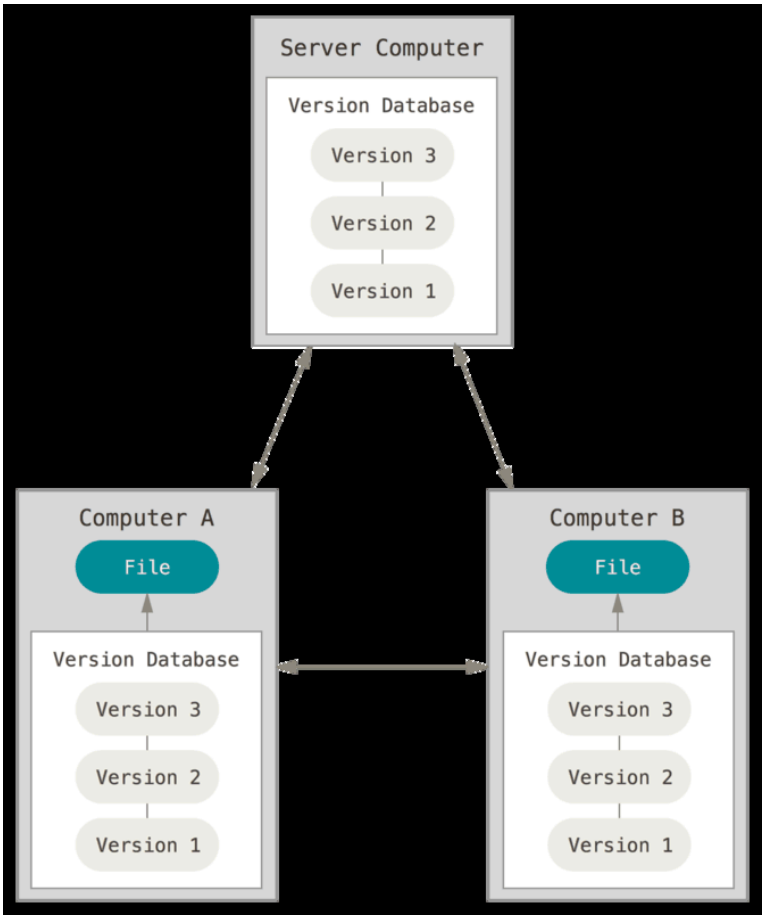
Alguns exemples de SCV centralitzats són:

- CVS (Current Versions System). Creat als anys 80 i amb llicència GNU, es considera el primer SCV centralitzat. Una característica distintiva és que el servidor només accepta actualitzacions dels fitxers que estiguen en l'última versió. Això implica que els usuaris han d'actualitzar les seues còpies locals abans de pujar-les al servidor.
- SVN. El projecte de Subversion va sorgir l'any 2000 amb l'objectiu de crear un sistema de control que solucionara les mancances i problemes de CVS, com per exemple:
  - Commits atòmics; en CVS un commit interromput pot deixar dades inconsistents.
  - La creació de branques és més eficient, amb complexitat constant a diferència de CVS, que és lineal (augmenta amb el nombre de branques).
  - Maneig de fitxers binaris com a tals; CVS els tracta com a fitxers de text.
  - Envia els increments dels fitxers en la comunicació client-servidor, en lloc dels fitxers complets com CVS.



3.3 Distribuït

Els sistemes de control de versions distribuïts (DVCS per les seues sigles en anglés) ofereixen solucions per als problemes que s'han mencionat. En un DVCS (com Git, Mercurial, Bazaar o Darcs), els clients no sols descarreguen l'última còpia instantània dels fitxers, sinó que es replica completament el repositori. D'aquesta manera, si un servidor deixa de funcionar i aquests sistemes estaven col·laborant a través d'ell, qualsevol dels repositoris disponibles en els clients pot ser copiat al servidor amb la finalitat de restaurar-lo. Cada clon és realment una còpia completa de totes les dades.



Alguns exemples de SCV distribuïts són:

- Mercurial. Creat en 2005, és un sistema de control de versions creat per Matt Mackall, escrit en Python i C. Destaca per la seua rapidesa i per ser multiplataforma i extensible.



- Bazaar. Distribuït i patrocinat per Canonical Ltd., dissenyat per facilitar la contribució en projectes de programari lliure i open source.



- Bitkeeper. Desenvolupada per l'empresa BitMover; durant un temps es va usar com a sistema de control de versions del codi de molts projectes de programari lliure, però sens dubte el més important va ser el del nucli de Linux. Encara que en un principi es tractava de programari propietari, en l'actualitat s'ha fet de codi obert adoptant la llicència lliure Apache 2.0.

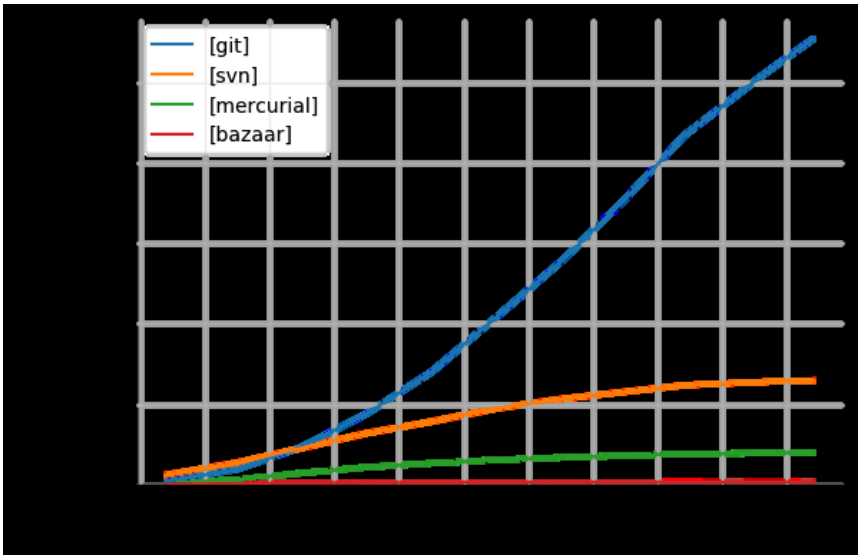


- Git. Sens dubte el SCV més utilitzat en l'actualitat. En 2005, la relació entre la comunitat que desenvolupava el nucli de Linux i la companyia que desenvolupava BitKeeper es va trencar i l'eina va deixar de ser oferida de manera gratuïta. Això va impulsar la comunitat de desenvolupament de Linux (i en particular Linus Torvalds, el creador de Linux) a desenvolupar la seua pròpia eina basada en algunes de les lliçons que van aprendre mentre usaven BitKeeper. Alguns dels objectius del nou sistema van ser els següents:
  - Velocitat.
  - Disseny senzill.
  - Gran suport per al desenvolupament no lineal (milers de branques paral·leles).
  - Completament distribuït.
  - Capaç de manejar grans projectes (com el nucli de Linux) eficientment (velocitat i grandària de les dades).



4. GIT

Com hem dit, Git és el sistema de control de versions més utilitzat en l'actualitat. Des del seu naixement en 2005, Git ha evolucionat i ha madurat per ser fàcil d'usar, sempre basant-se en una sèrie de característiques que li permeten destacar de la resta de solucions semblants que hem vist en el punt anterior.

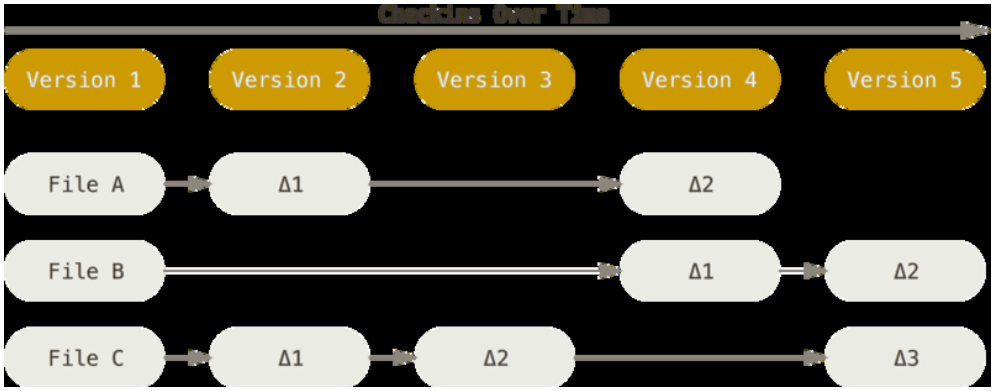


4.1 Fonaments de Git

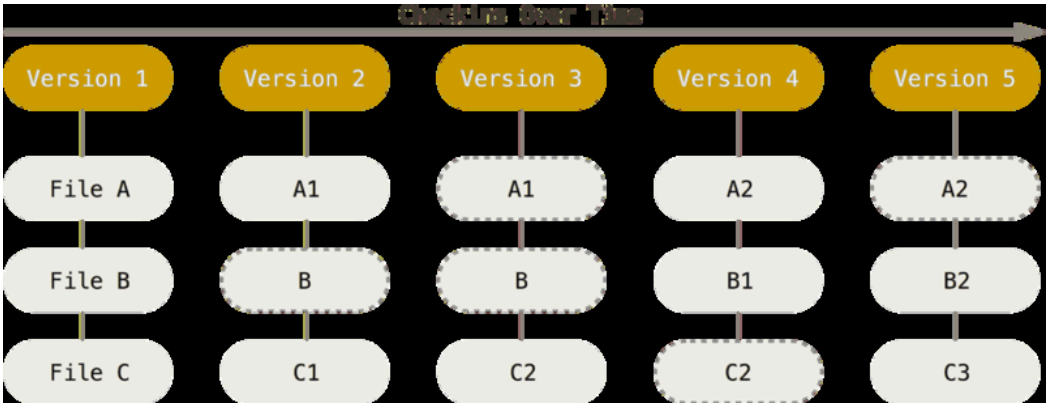
Entendre els fonaments de com funciona Git ens permetrà utilitzar l'eina de manera molt més efectiva. Vegem quins són:

Còpies instantànies, no diferències

Conceptualment, la majoria dels altres SCV manegen la informació que emmagatzemen com un conjunt de fitxers i les modificacions fetes a cadascun d'ells a través del temps.



Git no maneja ni emmagatzema les seues dades d'aquesta manera. **Git maneja les seues dades com un conjunt de còpies instantànies d'un sistema de fitxers miniatura**. **Cada** vegada que confirmes un **canvi**, o guardes l'estat del teu projecte en Git, ell bàsicament pren una **foto de l'aspecte de tots els teus fitxers en aquest moment** i guarda una referència a aquesta còpia instantània. Per ser eficient, **si els fitxers no s'han modificat Git no emmagatzema el fitxer de nou, sinó un enllaç al fitxer anterior idèntic** que ja té emmagatzemat. Git maneja les seues dades com una seqüència de còpies instantànies.



Gairebé totes les operacions són locals

La majoria de les operacions en Git només necessiten fitxers i recursos locals per funcionar. En no intervindre retards de xarxa com podria succeir en altres sistemes, Git resulta molt més ràpid. A més, aquesta característica permet que puguem continuar fent canvis i confirmant-los en el nostre servidor local sense necessitat d'estar connectats a la xarxa.

Integritat

Tot en Git és verificat mitjançant una suma de comprovació (checksum en anglés) abans de ser emmagatzemat, i és identificat a partir d'aquest moment mitjançant aquesta suma. Això significa que **és impossible canviar els continguts de qualsevol fitxer o directori sense que Git ho sàpiga**. No pots perdre informació durant la seua transmissió ni patir corrupció de fitxers sense que Git siga capaç de detectar-ho.

> Git guarda tot no per nom de fitxer, sinó pel valor hash dels seus continguts. Aquesta funcionalitat està integrada en Git al més baix nivell i és part integral de la seua filosofia.

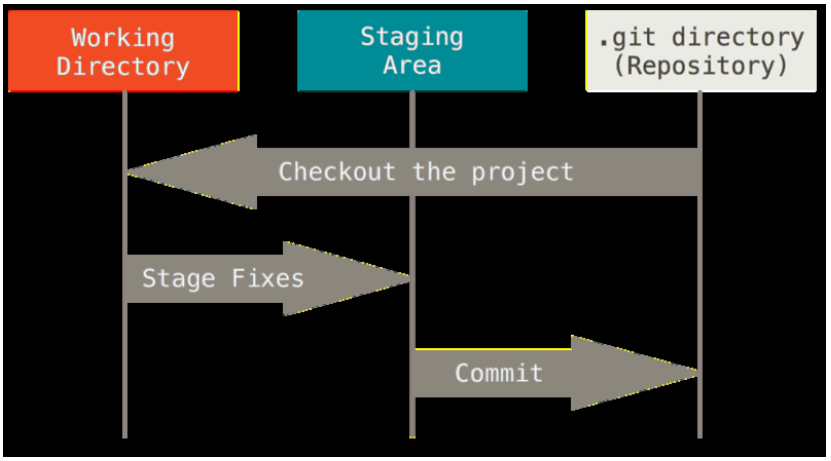
Generalment només afig informació

Quan realitzes accions en Git, gairebé totes elles només afigen informació a la base de dades de Git. És molt difícil aconseguir que el sistema faça alguna cosa que no es puga esmenar, o que d'alguna manera esborre informació.

Tres estats

Sens dubte és el fonament més important. Git té tres estats principals en els quals es poden trobar els teus fitxers:

- **Confirmat**: les dades estan emmagatzemades de manera segura en la teua base de dades local.
- **Modificat**: tot fitxer modificat però que encara no s'ha confirmat en la nostra base de dades.
- **Preparat**: tot fitxer modificat que s'ha marcat en la seua versió actual perquè vaja en la teua pròxima confirmació.



Així doncs, el flux de treball bàsic en Git és una cosa així:

1. Modifiquem una sèrie de fitxers en el nostre directori de treball.
2. Preparem els fitxers, afegint-los a la nostra àrea de preparació.
3. Confirmem els canvis, la qual cosa pren els fitxers tal com estan en l'àrea de preparació i emmagatzema aquesta còpia instantània de manera permanent en el nostre directori de Git.

4.2 Instal·lació

Git es troba disponible en la majoria de plataformes, incloent-hi Linux, Windows, MacOS i Solaris. En la pàgina oficial podem trobar l'enllaç per descarregar-lo per als diferents sistemes operatius.

La instal·lació és molt senzilla. Per exemple, en el cas de distribucions basades en Debian com Ubuntu, simplement hem d'executar la instrucció següent:

```
apt-get install git
```

Una vegada fet això, podem comprovar que efectivament la versió instal·lada és la correcta per mitjà de la instrucció:

```
git --version
```

4.3 Configuració inicial

Git ens permet personalitzar el nostre entorn. Per a això inclou una eina anomenada **git config** amb la qual podem establir variables de configuració com, per exemple, el nostre editor preferit per ser utilitzat quan necessitem introduir algun missatge de confirmació de canvis. Aquestes variables poden emmagatzemar-se en tres llocs distints:

- Fitxer **/etc/gitconfig**: conté valors per a tots els usuaris del sistema i tots els seus repositoris. Si passem l'opció --system a git config, llig i escriu específicament en aquest fitxer.

  ```
  git config --system core.editor emacs
  ```

- Fitxer **~/.gitconfig o ~/.config/git/config**: aquest fitxer és específic del nostre usuari. Podem fer que Git llija i escriga específicament en aquest fitxer passant l'opció --global.

  ```
  git config --global user.name "Pepe"
  ```

- Fitxer config en el directori de Git (és a dir, **.git/config**) del repositori que estiguem utilitzant actualment: aquest fitxer és específic del repositori actual.

> Cada nivell sobreescriu els valors del nivell anterior, per la qual cosa els valors de .git/config tenen preferència sobre els de /etc/gitconfig.

Una vegada realitzats els canvis en la configuració que desitgem, podem comprovar que aquests s'han emmagatzemat correctament. Per a això, només hem d'executar:

```
git config --list
```

I si només volem verificar el valor d'una determinada propietat:

```
git config user.name
```

4.4 Obtenció d'un repositori

El primer pas per començar a treballar amb Git en local una vegada instal·lat és disposar d'un repositori sobre el qual emmagatzemar els nostres canvis. En aquesta secció veurem les dues principals formes de poder obtindre'n un: inicialitzar-lo nosaltres o clonar-ne un que ja existisca prèviament. En tots dos casos, caldrà haver creat prèviament en remot un repositori destí (per exemple en GitHub).

Inicialitzar un repositori

Transformar un directori del nostre sistema en el qual emmagatzemem el contingut del nostre projecte en un repositori Git és molt senzill. Només hem de situar-nos en aquesta ubicació i usar l'ordre següent:

```
git init
```

Això crea un subdirectori nou anomenat **.git**, el qual conté tots els fitxers necessaris del repositori.

```
$ ls -F1
HEAD
config*
description
hooks/
info/
objects/
refs/
```

- La carpeta *objects* guarda el contingut de la teua base de dades.
- La carpeta *refs* guarda els apuntadors a les confirmacions de canvis (commits).
- El fitxer *HEAD* apunta a la branca que tingues activa (checked out) en aquest moment.
- El fitxer *index* és on Git emmagatzema la informació sobre la teua àrea de preparació (staging area).
- El fitxer *config* conté les opcions de configuració específiques del projecte.
- La carpeta *info* guarda un fitxer global d'exclusió amb els patrons a ignorar a més dels presents en el fitxer .gitignore.
- El fitxer *description* s'utilitza només en el programa GitWeb.

> Una manera d'eliminar el control de versions d'un directori consisteix a eliminar la carpeta .git del seu interior.

Després d'executar l'ordre `git init` sobre el nostre directori, necessitarem una sèrie de passos extra per poder connectar aquest amb el repositori remot que, com hem dit, hem d'haver creat prèviament. Tots aquests passos sol exposar-los GitHub en el moment que creem un repositori des de zero i solen ser:

…or create a new repository on the command line

```
echo "# comandosInicialesGit" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M master
git remote add origin https://github.com/dawEstacio/comandosInicialesGit.git
git push -u origin master
```

Si analitzem els passos que ens suggereix GitHub veiem que:

- En primer lloc, creem el fitxer README.md (o qualsevol altre).
- Realitzem el git init que hem comentat.
- Fem l'add i el commit del fitxer README.md.
- Creem una branca master (git branch -M master).
- Connectem el nostre directori amb el repositori de GitHub (**git remote** add origin ...).

- Fem el push de tots els canvis locals al repositori remot.

Clonar un repositori

La segona de les formes d'obtindre un repositori Git consisteix a obtindre una còpia o clon d'un que ja existisca. En aquest cas, la instrucció que haurem d'executar serà **git clone [url]**. Per exemple:

```
git clone https://github.com/dawEstacio/comandosInicialesGit.git
```

L'execució de l'ordre anterior crearà un directori anomenat *comandosInicialesGit*, inicialitza un directori .git en el seu interior, descarrega tota la informació d'aquest repositori i trau una còpia de treball de l'última versió.

```
pepe@pepe-Lenovo-ideapad-530S-14IKB:~$ git clone https://github.com/dawEstacio/comandosInicialesGit.git
Clonando en 'comandosInicialesGit'...
warning: Pareces haber clonado un repositorio sin contenido.
```

Si volem que el procés de clonació genere un directori amb un nom distint al del projecte (en l'exemple anterior netflix.github.com.git), podem fer-ho incloent el nom desitjat com a segon paràmetre del git clone.

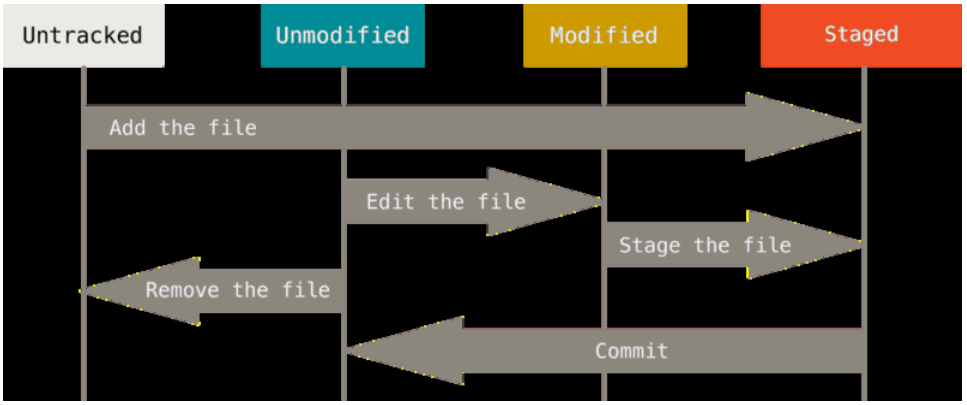
```
git clone https://github.com/dawEstacio/comandosInicialesGit.git
misComandos
```

L'execució d'aquesta ordre farà el mateix que l'anterior però el directori on descarregarà tota la informació serà *misComandos*.

4.5 Ordres bàsiques

Revisar l'estat del repositori

Com hem vist, un dels fonaments principals de Git consisteix a diferenciar els distints fitxers emmagatzemats segons el seu estat. Per aquest motiu, és interessant conéixer de quina forma podem esbrinar quins elements del nostre projecte estan rastrejats i quins no. **Els fitxers rastrejats (tracked files en anglés) són tots aquells fitxers que estaven en l'última instantània del projecte**; poden ser fitxers sense modificar, modificats o preparats. Els fitxers sense rastrejar són tots els altres: qualsevol altre fitxer en el teu directori de treball que no estava en la teua última instantània i que no està en l'àrea de preparació (staging area).



L'eina principal per determinar en quin estat estan els fitxers d'un projecte és l'ordre **git status**. Si executem aquesta ordre immediatament després de clonar un repositori, obtenim una eixida semblant a la següent:

```
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git status
En la rama master

No hay commits todavía

no hay nada para confirmar (crea/copia archivos y usa "git add" para hacerles seguimiento)
```

L'ordre t'indica en quina branca estàs i t'informa sobre què ha variat respecte a la mateixa branca en el servidor.

Imaginem que afegim un nou fitxer (README) en el nostre servidor. Si tornem a consultar l'estat del repositori, veurem com l'eixida de git status ens informa que existeix un nou fitxer no rastrejat.

```
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ echo "Hola mundo" > primerFichero.txt
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git status
En la rama master

No hay commits todavía

Archivos sin seguimiento:
  (usa "git add <archivo>..." para incluirlo a lo que se será confirmado)

        primerFichero.txt

no hay nada agregado al commit pero hay archivos sin seguimiento presentes (usa "git add" para hacerles seguimient
o)
```

Si tot seguit rastregem el fitxer README en el nostre servidor (mitjançant la instrucció **git add** que veurem a continuació) i consultem l'estat de nou, obtindrem una eixida semblant a la següent:

```
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git add primerFichero.txt
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git status
En la rama master

No hay commits todavía

Cambios a ser confirmados:
  (usa "git rm --cached <archivo>..." para sacar del área de stage)

        nuevo archivo:  primerFichero.txt
```

Rastrejar fitxers (git add)

En els exemples de consulta d'estat que hem vist en el punt anterior, hem fet ús de la instrucció *git add*. Així doncs, **git add** ens permet rastrejar fitxers (passar-los a la stage area) del nostre directori de treball indicant així que s'han realitzat els canvis oportuns sobre aquests i que formaran part del pròxim commit al nostre servidor local.

La sintaxi de la instrucció git add admet com a paràmetre tant el nom d'un fitxer en concret, com un patró. Per tant, podem executar:

```
git add README.md (només rastreja el fitxer README.md)
```

o bé:

```
git add . (rastreja tots els fitxers del directori)
```

Algunes opcions interessants de git add són:

- **-u**: només afig a l'índex aquells fitxers que ja estan sent monitorats per git.
- **-n**: aquesta opció és molt pràctica ja que ens mostrarà en pantalla el que l'ordre git-add faria **sense actualitzar l'índex**.

Si volem eliminar de la nostra staged area algun fitxer podem utilitzar l'ordre `git reset HEAD nomFitxer` per tornar-lo al seu estat de no confirmat. Això sí, ha d'existir algun commit previ en el nostre repositori perquè existisca el HEAD.

Eliminar fitxers (git rm)

Per eliminar fitxers de Git, hem d'eliminar-los dels nostres fitxers rastrejats (de l'àrea de preparació) i després confirmar. **No n'hi ha prou amb eliminar-los físicament del directori de treball**. Per a això existeix l'ordre **git rm**, que s'encarrega d'eliminar el fitxer del nostre directori de treball, i a més l'afig a l'àrea de staged files. La diferència entre aquesta ordre i eliminar directament el fitxer és que si només l'eliminem físicament apareixerà en l'àrea d'unstaged files.

```
$ rm PROJECTS.md
$ git status
On branch master
Your branch is up-to-date with 'origin/master'.
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

        deleted:    PROJECTS.md

no changes added to commit (use "git add" and/or "git commit -a")
```

```
$ git rm PROJECTS.md
rm 'PROJECTS.md'
$ git status
On branch master
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

    deleted:    PROJECTS.md
```

L'ordre **git rm** resulta molt interessant quan volem canviar el nom d'un fitxer ja rastrejat. Amb un sol pas (per exemple `git rm README.md README`), fem l'equivalent a:

- Canviar físicament el nom del fitxer `mv README.md README`
- Deixar de rastrejar el fitxer antic `git rm README.md`
- Rastrejar el fitxer nou `git add README`

Igual que passava amb l'ordre git reset, podem utilitzar `git rm --cached nomFitxer` per eliminar fitxers de la nostra àrea de preparació. No obstant això, cal tindre en compte que:

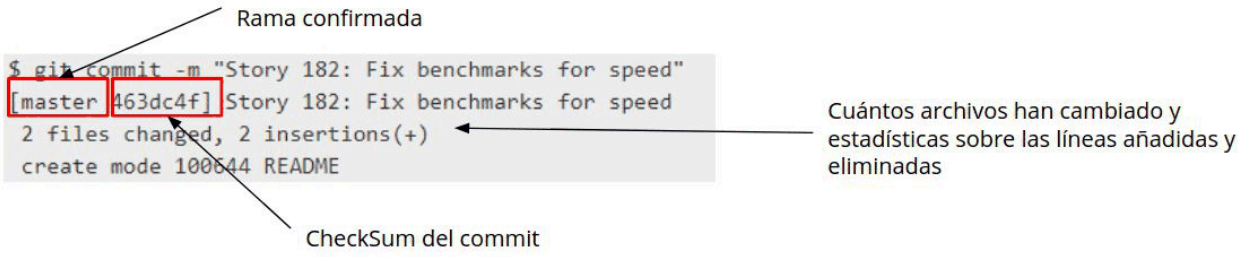
- git rm --cached elimina el fitxer de l'índex però el deixa en el directori de treball. Això li indica a Git que ja no desitja rastrejar el fitxer.
- git reset HEAD deixa el fitxer com un fitxer de seguiment en l'índex, però les modificacions emmagatzemades en memòria cau en l'índex es perden.

Confirmar canvis (git commit)

Una vegada tenim els nostres fitxers rastrejats, és moment d'incloure'ls en el nostre repositori local. Amb aquesta finalitat, utilitzarem la instrucció **git commit**. La forma més senzilla consisteix a executar aquesta ordre sense cap tipus de paràmetre. Automàticament, Git obrirà un editor (que com hem vist podem configurar) per poder introduir els comentaris associats a la confirmació d'aquests canvis.

Aquests dos passos (realitzar el commit i escriure el comentari associat) poden executar-se al mateix temps si afegim l'opció -m com en l'exemple següent:

```
git commit -m "Comentari associat a la confirmació de canvis"
```



Altres opcions interessants de git commit són:

- **--amend**: permet afegir nous fitxers a l'últim commit realitzat.
- **-a**: permet, amb un sol pas, passar a preparats els fitxers modificats o eliminats que ja estiguen en l'àrea de preparació, i realitzar el commit.

Revisar diferències entre versions (git diff)

L'ordre **git diff** s'utilitza quan desitgem veure les diferències entre distintes versions dels nostres fitxers. Les comparacions que més se solen utilitzar són:

- git diff (sense cap paràmetre): revisa les diferències entre el nostre entorn de treball i la nostra staged area.
- git diff --staged o git diff --cached: revisa les diferències entre la nostra staged area i l'última confirmació o commit.
- git diff nomBranca1 nomBranca2: revisa diferències entre dues branques distintes.

Un exemple d'eixida de l'ordre git diff seria el que es mostra a continuació. En aquest cas, en el nostre entorn de treball tenim el fitxer CONTRIBUTING.md que encara no hem afegit a la stage area:

```
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/Escritorio/primerRepositorio_1$ git diff
diff --git a/fichero1.txt b/fichero1.txt
index 1391490..04966b2 100644
--- a/fichero1.txt
+++ b/fichero1.txt
@@ -1 +1 @@
-Soy el usuario 1 con vcode (Mis cambios son los primeros) y quiero mi cambio del usuario 2
+Soy el usuario 1 con vcode haciendo cambios (Mis cambios son los primeros) y quiero mi cambio del usuario 2
```

Revisar historial de canvis (git log)

Després d'haver fet diverses confirmacions, o si has clonat un repositori que ja tenia un històric de confirmacions, probablement vulgues mirar arrere per veure quines modificacions s'han dut a terme. L'eina més bàsica i potent per fer això és l'ordre git log.

En cas de no passar cap paràmetre a la crida, **git log** ens mostra les confirmacions fetes sobre aquest repositori en ordre cronològic invers. És a dir, *les confirmacions més recents es mostren al principi*.

Un exemple d'eixida de l'ordre git log seria el següent:

```
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git log
commit c93df2609c3eac015a471389d299aa1d112cb2e5 (HEAD -> master)
Author: Pepe <vidal_jossal@iestacio.com>
Date:   Mon Sep 14 10:31:59 2020 +0200

    Creado fichero primerFichero.txt
```

Algunes opcions interessants de git log són:

- git log -p: mostra les diferències introduïdes en cada confirmació.
- git log -valorNumèric: fa que es mostren únicament les valorNumèric últimes entrades de l'historial.
- git log --oneline: mostra el log en una sola línia, la qual cosa permet que veiem més quantitat de commits en la pantalla i facilita molt seguir la seqüència, en lloc d'haver de veure un munt de pàgines de commits.

L'ordre **git shortlog** s'utilitza per resumir l'eixida de git log. Pren moltes de les mateixes opcions que l'ordre git log però, en lloc d'enumerar tots els commits, presentarà un resum dels commits agrupats per autor.

```
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git shortlog
Pepe (1):
      Creado fichero primerFichero.txt
```

Ignorar fitxers (.gitignore)

Aquesta no és una ordre de git com a tal, sinó una opció que Git ens ofereix. Pot ser que existisquen determinats fitxers en el nostre espai de treball que no volem que Git rastrege. Doncs bé, en aquests casos tenim l'opció de crear un fitxer **.gitignore** on llistarem els patrons que no volem que Git tinga en compte.

Crear un fitxer .gitignore abans de començar a treballar és generalment una bona idea, perquè així evitem confirmar accidentalment fitxers que en realitat no volem incloure en el nostre repositori.

Les regles sobre els patrons que pots incloure en el fitxer .gitignore són les següents:

- Ignorar les línies en blanc i aquelles que comencen amb #.
- Acceptar patrons glob estàndard (https://rico-schmidt.name/pymotw-3/glob/).
- Els patrons poden acabar en barra (/) per especificar un directori.
- Els patrons poden negar-se si s'afig al principi el signe d'exclamació (!).

Un exemple de fitxer .gitignore podria ser:

```
# ignora los archivos terminados en .a
*.a

# pero no lib.a, aun cuando había ignorado los archivos terminados en .a en la línea anterior
!lib.a

# ignora unicamente el archivo TODO de la raiz, no subdir/TODO
/TODO

# ignora todos los archivos del directorio build/
build/

# ignora doc/notes.txt, pero no este: doc/server/arch.txt
doc/*.txt

# ignora todos los archivos .txt del directorio doc/
doc/**/*.txt
```

> GitHub manté una extensa llista de fitxers .gitignore adequats a dotzenes de projectes i llenguatges en el següent enllaç.

4.6 Ordres avançades

git push

Quan tenim un projecte que volem compartir, hem d'enviar-lo a un servidor. L'ordre per fer-ho és simple: **git push [nom-remot] [nom-branca-local]**. Per exemple, si volem enviar la nostra branca master al nostre servidor origin, podem executar l'ordre següent:

```
git push origin master
```

Aquesta ordre *només funciona si vas clonar d'un servidor sobre el qual tenim permisos d'escriptura i si ningú més ha enviat dades pel mig*. Si algú més clona el mateix repositori i envia informació abans que nosaltres, el nostre enviament serà rebutjat. En aquest cas, haurem de portar-nos el seu treball i combinar-lo amb el nostre abans de poder enviar dades al servidor.

```
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git push origin master
Username for 'https://github.com': dawEstacio
Password for 'https://dawEstacio@github.com':
Contando objetos: 6, listo.
Delta compression using up to 8 threads.
Comprimiendo objetos: 100% (2/2), listo.
Escribiendo objetos: 100% (6/6), 502 bytes | 502.00 KiB/s, listo.
Total 6 (delta 0), reused 0 (delta 0)
To https://github.com/dawEstacio/comandosInicialesGit.git
 * [new branch]      master -> master
```

git fetch

Pot ser que el contingut del servidor remot siga modificat mentre nosaltres treballem en el nostre servidor local i necessitem veure quins han sigut els canvis efectuats per poder ajuntar-los amb els nostres. En aquests casos podem optar per dues opcions:

- Obtindre en un primer pas els canvis remots però no ajuntar-los (fusionar-los) amb els nostres i en un segon pas realitzar la unió.
- Obtindre i combinar els canvis remots amb un únic pas.

Per al primer dels casos, Git ens ofereix l'ordre **git fetch [nom branca remota]**. Una vegada executada, l'ordre anirà al projecte remot i es portarà totes les dades que encara no tenim en el nostre servidor local. Després de fer això, tindrem referències a totes les branques del remot, les quals podem combinar i inspeccionar quan vulgues.

En l'exemple següent, un segon usuari ha creat un fitxer (ficheroUsuario2.txt) del qual nosaltres encara no disposem. Si realitzem un git fetch de la branca origen i posteriorment consultem el contingut del directori, veiem com continuem sense disposar-ne. El que sí que podem observar és que s'ha creat una nova branca (remotes/origin/master).

```
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git fetch origin
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0
Desempaquetando objetos: 100% (3/3), listo.
Desde https://github.com/dawEstacio/comandosInicialesGit
   eb9462a..c6d55c3  master     -> origin/master
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git branch -a
* master
  remotes/origin/master
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ ls
primerFichero.txt
```

En canvi, si consultem el contingut de la nova branca veiem que el fitxer creat per l'usuari 2 sí que hi és:

```
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git checkout remotes/origin/master
Nota: actualizando el árbol de trabajo 'remotes/origin/master'.

Te encuentras en estado 'detached HEAD'. Puedes revisar por aquí, hacer
cambios experimentales y confirmarlos, y puedes descartar cualquier
commit que hayas hecho en este estado sin impactar a tu rama realizando
otro checkout.

Si quieres crear una nueva rama para mantener los commits que has creado,
puedes hacerlo (ahora o después) usando -b con el comando checkout. Ejemplo:

  git checkout -b <nombre-de-nueva-rama>

HEAD está ahora en c6d55c3 Creación de fichero por el usuario 2
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ ls
ficheroUsuario2.txt  primerFichero.txt
```

Per tant, **és important destacar que l'ordre git fetch només porta dades al nostre repositori local. Ni les combina automàticament amb el nostre treball ni modifica el treball que portem fet**. La combinació amb el nostre treball hem de fer-la manualment quan estiguem preparats.

En el nostre cas hauríem de fer un merge (que veurem més endavant però que bàsicament afig els canvis d'una branca en una altra) de la branca nova que s'ha creat a la branca master per poder disposar dels canvis de l'usuari 2.

```
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git checkout master
La posición previa de HEAD era c6d55c3 Creación de fichero por el usuario 2
Cambiado a rama 'master'
Tu rama está detrás de 'origin/master' por 1 commit, y puede ser avanzada rápido.
  (usa "git pull" para actualizar tu rama local)
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ ls
primerFichero.txt
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git merge remotes/origin/master
Actualizando eb9462a..c6d55c3
Fast-forward
 ficheroUsuario2.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 ficheroUsuario2.txt
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ ls
ficheroUsuario2.txt  primerFichero.txt
```

git pull

Imaginem ara que volem actualitzar el nostre repositori local amb els possibles canvis que s'hagen pogut produir en el repositori remot. Si treballem en un mateix repositori amb més gent, aquest és un pas imprescindible per mantindre la nostra còpia local del repositori al dia. De fet, si existeixen canvis en el repositori remot, Git no ens deixarà pujar canvis al repositori fins que hàgem descarregat la versió més recent i l'hàgem fusionada amb la nostra versió local. Fins i tot si treballem sols, és possible que pugem canvis des de més d'un ordinador, per la qual cosa saber com descarregar els canvis del repositori remot també resulta útil.

La forma més senzilla d'actualitzar el repositori local és mitjançant l'ordre **git pull**:

```
git pull
```

A continuació es mostra un exemple de l'ordre git pull. En aquest cas, previ a l'execució d'aquesta instrucció, un altre usuari amb permisos ha modificat un fitxer del repositori. En executar el git pull

veiem com es descarrega el canvi en el nostre repositori local.

```
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git pull
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0
Desempaquetando objetos: 100% (3/3), listo.
Desde https://github.com/dawEstacio/comandosInicialesGit
   c6d55c3..9ab1f12  master     -> origin/master
Actualizando c6d55c3..9ab1f12
Fast-forward
 ficheroUsuario2.txt | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

A diferència de git fetch, git pull descarrega els canvis i intenta fusionar-los sobre la nostra branca de treball.

És possible, no obstant això, que en git ens mostre un error com el següent:

```
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git pull
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (1/1), done.
remote: Total 3 (delta 1), reused 3 (delta 1), pack-reused 0
Desempaquetando objetos: 100% (3/3), listo.
Desde https://github.com/dawEstacio/comandosInicialesGit
   9ab1f12..6e6bc23  master     -> origin/master
Auto-fusionando primerFichero.txt
CONFLICTO (contenido): Conflicto de fusión en primerFichero.txt
Fusión automática falló; arregle los conflictos y luego realice un commit con el resultado.
```

Això ocorre quan git no pot realitzar el merge després d'haver-se portat del remot tots els canvis. En aquest cas hem realitzat canvis sobre el *primerFichero.txt*. Després d'aquests canvis (sobre els quals hem fet commit), hem intentat realitzar el pull per actualitzar el nostre directori local. El que ha passat és que, com que un altre usuari ja havia pujat un nou canvi sobre *primerFichero.txt* que nosaltres no teníem en local, git no sap com combinar tots dos i ens delega aquesta tasca a nosaltres.

Si editem el contingut de *primerFichero.txt*, veurem que té l'aspecte següent:

```
<<<<<<< HEAD
Hola mundo modificado por el primer usuario
=======
Hola mundo modificado por el usuario 2
>>>>>>> 6e6bc2364bf718f8e63ec1929048466ec2ef8bbf
```

Veiem que git ens ha modificat el contingut de *primerFichero.txt*. La primera línia (<<<<<<< HEAD) indica que els canvis que hi ha fins a la línia ====== són els que nosaltres hem realitzat en local. La part posterior a aquesta marca són els canvis importats del repositori remot. Ara, la nostra tasca consistirà a realitzar la combinació de totes dues parts per deixar *primerFichero.txt* amb el contingut definitiu i sense cap tipus de marques que puguen indicar a git que existeixen conflictes. Una vegada fet això i tal com ens marcava el git pull, realitzarem el git add i git commit del fitxer.

4.7 Treballar amb branques

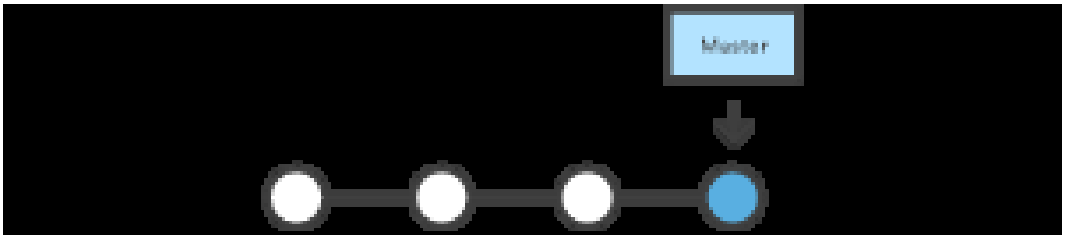
Les branques són una de les principals utilitats que disposem en qualsevol sistema de control de versions modern per portar un millor control del codi. Es tracta d'una bifurcació de l'estat del codi que crea un nou camí de cara a l'evolució del codi, en paral·lel a altres branques que es puguen generar.

Les branques ens poden servir per a molts casos d'ús. Per exemple, per a la creació d'una funcionalitat que vulguem integrar en un programa i per a la qual no volem que la branca principal es veja afectada. Aquesta funció experimental es pot realitzar en una branca independent, de manera que, encara que tardem diversos dies o setmanes a acabar-la, no afecte la producció del codi que tenim en la branca principal i que romandrà estable.

El treball amb branques resulta molt còmode en el desenvolupament de projectes, perquè és possible que totes les branques creades evolucionen al mateix temps, i el desenvolupador pot passar d'una branca a una altra en qualsevol moment segons les necessitats del projecte. Si en un moment donat el treball amb una branca ens ha resultat interessant, útil i es troba estable, llavors podem fusionar branques per incorporar les modificacions del projecte en la seua branca principal.

git branch

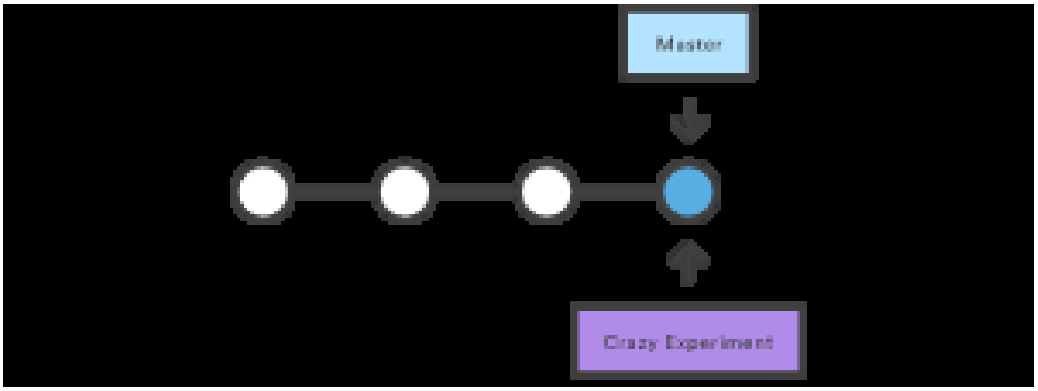
És important comprendre que les branques són només punters a les confirmacions. Quan crees una branca, tot el que Git ha de fer és crear un nou punter; no modifica el repositori de cap altra forma. Si comences amb un repositori que té aquest aspecte:



I, a continuació, crees una branca amb l'ordre següent:

```
git branch crazy-experiment
```

L'historial del repositori no es modificarà. Tot el que necessites és un nou punter de la confirmació actual:



```
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git branch crazy-experiment
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git branch
  crazy-experiment
* master
```

Cal tindre en compte que aquesta ordre només crea la nova branca. Per començar a afegir confirmacions, necessitarem seleccionar-la amb l'ordre **git checkout** que ara veurem i, a continuació, utilitzar les ordres estàndard git add i git commit.

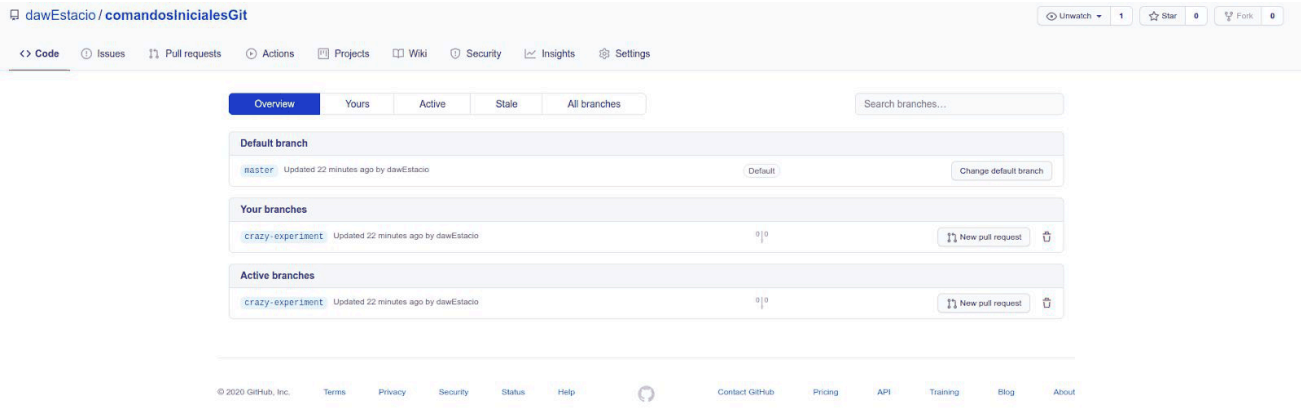
Creació de branques remotes

Ara que ja hem creat una branca en local, necessitem saber com pujar-la al nostre repositori remot perquè la resta d'usuaris puga tindre-hi accés. Per a això, n'hi haurà prou amb executar l'ordre push següent:

```
git push -u origin <nom_branca_creada>
```

Per ara, totes les branques són locals. L'ordre git branch també funciona amb branques remotes. Per treballar en branques remotes, primer cal configurar un repositori remot i afegir-lo a la configuració del repositori local.

```
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git push -u origin crazy-experiment
Username for 'https://github.com': dawEstacio
Password for 'https://dawEstacio@github.com':
Total 0 (delta 0), reused 0 (delta 0)
remote:
remote: Create a pull request for 'crazy-experiment' on GitHub by visiting:
remote:      https://github.com/dawEstacio/comandosInicialesGit/pull/new/crazy-experiment
remote:
To https://github.com/dawEstacio/comandosInicialesGit.git
 * [new branch]      crazy-experiment -> crazy-experiment
Rama 'crazy-experiment' configurada para hacer seguimiento a la rama remota 'crazy-experiment' de 'origin'.
```



Eliminació de branques

Una vegada que hàgem acabat de treballar en una branca i l'hàgem fusionada amb el codi base principal, podem eliminar la branca sense perdre cap història executant:

```
git branch -d <nom_branca_creada>
```

```
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git branch -d crazy-experiment
Eliminada la rama crazy-experiment (era a0b91a5)..
```

No obstant això, si la branca no s'ha fusionat, l'ordre anterior mostrarà un missatge d'error:

```
error: The branch 'crazy-experiment' is not fully merged.
If you are sure you want to delete it, run 'git branch -D crazy-experiment'.
```

Això ens protegeix davant la pèrdua d'accés a una línia de desenvolupament completa. Si realment volem eliminar la branca (per exemple, si es tracta d'un experiment fallit), podem usar l'indicador -D (en majúscula):

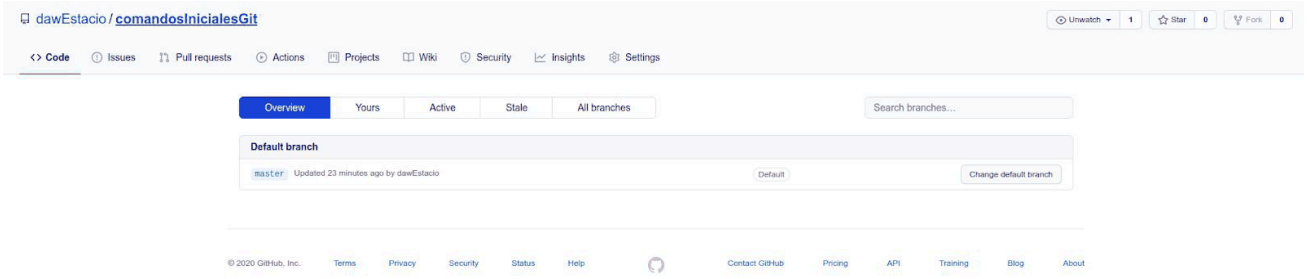
```
git branch -D <nom_branca_creada>
```

Aquesta ordre elimina la branca independentment del seu estat i sense avisos previs, així que hem d'usar-la amb cura.

Les ordres anteriors eliminaran una còpia local de la branca però continuarà existint en el repositori remot. Per eliminar una branca remota, executa aquestes ordres:

```
git push origin --delete <nom_branca_creada>
```





git checkout

L'ordre git checkout ens permet desplaçar-nos entre les branques creades per git branch. En executar l'operació de canvi de branca, s'actualitzen els fitxers en el directori de treball per reflectir la versió emmagatzemada en aquesta branca i s'indica a Git que registre totes les confirmacions noves en aquesta branca. Podem pensar-hi com una forma de seleccionar en quina línia de desenvolupament anem a treballar.

De vegades, l'ordre git checkout pot confondre's amb git clone. La diferència entre totes dues ordres rau en el fet que la segona extrau codi d'un repositori remot, mentre que la primera canvia entre versions de codi que ja es troben en el sistema local.

L'ordre git checkout va de la mà de git branch que ja hem vist. Una vegada creada una branca amb git branch, podem usar git checkout <nom branca> per canviar a aquesta branca.

```
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git branch -a
* master
  remotes/origin/master
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git checkout remotes/origin/master
Nota: actualizando el árbol de trabajo 'remotes/origin/master'.

Te encuentras en estado 'detached HEAD'. Puedes revisar por aquí, hacer
cambios experimentales y confirmarlos, y puedes descartar cualquier
commit que hayas hecho en este estado sin impactar a tu rama realizando
otro checkout.

Si quieres crear una nueva rama para mantener los commits que has creado,
puedes hacerlo (ahora o después) usando -b con el comando checkout. Ejemplo:

  git checkout -b <nombre-de-nueva-rama>

HEAD está ahora en a0b91a5 Realizado el merge de los cambios
```

L'ordre git checkout accepta l'argument -b, que actua com un pràctic mètode que crearà la nova branca i canviarà a ella a l'instant.

```
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git checkout -b ramaProvisional
Cambiado a nueva rama 'ramaProvisional'
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git branch
  master
* ramaProvisional
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git branch -a
  master
* ramaProvisional
  remotes/origin/master
```

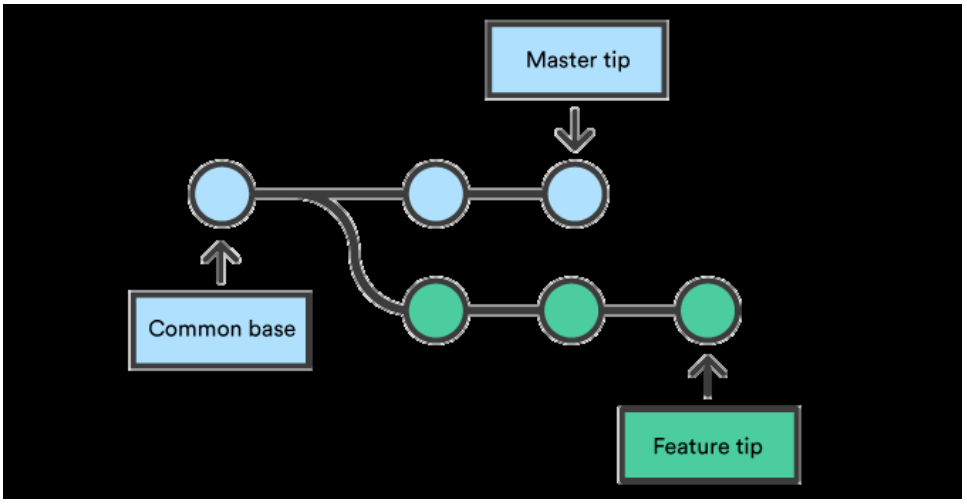
De manera predeterminada, git checkout -b basarà la branca new-branch en el HEAD actual. No obstant això, git checkout pot combinar-se amb un paràmetre opcional per a branques addicionals. En l'exemple següent, s'afig <existing-branch>, que basa new-branch en existing-branch en lloc del HEAD actual.

```
git checkout -b <nova_branca> <branca_existent>
```

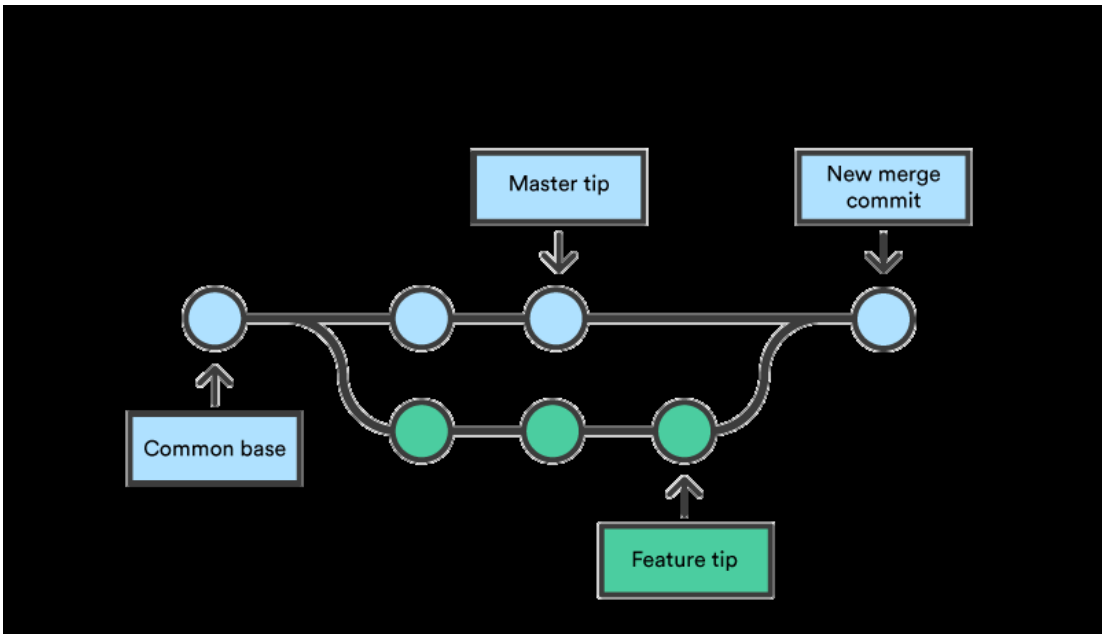
git merge

La fusió és la forma que té Git de tornar a unir un historial bifurcat. L'ordre **git merge** permet prendre les línies independents de desenvolupament creades per git branch i integrar-les en una sola branca.

Suposem que tenim una branca de funcionalitat nova que es basa en la branca master. Ara, volem fusionar aquesta branca de funcionalitat amb la master.



En invocar l'ordre `git merge feature_tip`, la branca de funcionalitat especificada es fusionarà amb la branca actual, la qual assumirem que és la master. Git determinarà l'algorisme de fusió automàticament.



Abans d'executar una fusió, hi ha un parell de passos de preparació que cal dur a terme amb la finalitat de garantir que la fusió es realitze sense problemes.

- Assegurar-nos que estem en la branca receptora. Executant `git status` podem comprovar si el HEAD apunta a la branca de fusió-recepció correcta. En cas necessari, executarem `git checkout <branca_desitjada>` per canviar a la branca de recepció.
- Recuperar les últimes confirmacions remotes. Hem d'assegurar-nos que la branca de recepció i la branca de fusió estan actualitzades amb els últims canvis remots. Per a això, com hem vist, podem executar `git fetch` + `git merge` o `git pull`.

Una vegada adoptats els passos comentats anteriorment de "preparació per a la fusió", és possible iniciar una fusió mitjançant l'execució de `git merge <branch name>` on <branch name> és el nom de la branca que es fusionarà amb la branca de recepció.

```
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git branch -a
  master
* ramaProvisional
  remotes/origin/master
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git checkout master
Cambiado a rama 'master'
Tu rama está actualizada con 'origin/master'.
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git branch -a
* master
  ramaProvisional
  remotes/origin/master
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git merge ramaProvisional
Actualizando a0b91a5..fafae58
Fast-forward
 primerFichero.txt | 2 ++
 1 file changed, 2 insertions(+)
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$
```

Si les dues branques que tractes de fusionar han canviat la mateixa part del mateix fitxer, Git no podrà esbrinar quina versió utilitzar. Quan això ocorre, Git s'atura just abans de la confirmació de fusió perquè puguem resoldre els conflictes manualment.

La major part del procés de fusió de Git consisteix a utilitzar el conegut flux de treball d'edició, preparació i confirmació per resoldre els conflictes de fusió. Quan s'observa un conflicte de fusió, l'execució de l'ordre git status mostra quins fitxers s'han de resoldre.

```
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git merge ramaProvisional
Auto-fusionando primerFichero.txt
CONFLICTO (contenido): Conflicto de fusión en primerFichero.txt
Fusión automática falló; arregle los conflictos y luego realice un commit con el resultado.
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git status
En la rama master
Tu rama está adelantada a 'origin/master' por 2 commits.
  (usa "git push" para publicar tus commits locales)

Tienes rutas no fusionadas.
  (arregla los conflictos y corre "git commit"
  (usa "git merge --abort" para abortar la fusion)

Rutas no fusionadas:
  (usa "git add <archivo>..." para marcar una resolución)

        ambos modificados:     primerFichero.txt

sin cambios agregados al commit (usa "git add" y/o "git commit -a")
```

4.8 Etiquetatge

Com molts SCV, Git té la possibilitat d'etiquetar punts específics de l'historial com a importants. Aquesta funcionalitat s'usa típicament per marcar versions de llançament (v1.0, per exemple).

Llistar etiquetes

Llistar les etiquetes disponibles en Git és senzill. Simplement escrivim **git tag**:

```
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git tag
v1.0
```

Aquesta ordre llista les etiquetes en ordre alfabètic.

També podem buscar etiquetes amb un patró particular. Imaginem que el nostre repositori conté multitud d'etiquetes. Si només ens interessa veure les de la versió 1., podem executar:

```
git tag -l 'v1.*'
```

Crear etiquetes

Git utilitza **dos tipus principals d'etiquetes**:

- Una etiqueta **lleugera** és molt semblant a una branca que no canvia; simplement és un punter a un commit específic. Per crear una etiqueta lleugera, no hem de passar les opcions -a, -s ni -m:

  ```
  git tag v1.4
  ```

- Les etiquetes **anotades** es guarden en la base de dades de Git com a objectes sencers. Tenen un checksum; contenen el nom de l'etiquetador, correu electrònic i data; tenen un missatge associat; i poden ser signades i verificades amb GNU Privacy Guard (GPG). Normalment es recomana crear etiquetes anotades, de manera que tinguem tota la informació.

  ```
  git tag -a v1.4 -m "Tag de la versió 1.4"
  ```

  L'opció -m especifica el missatge de l'etiqueta, el qual és guardat junt amb ella. Si no especifiquem el missatge d'una etiqueta anotada, Git obrirà l'editor de text perquè l'escrivim.

> Si una etiqueta coincideix amb algun dels commits del nostre repositori, podem veure-ho en l'eixida de l'ordre *git log*.



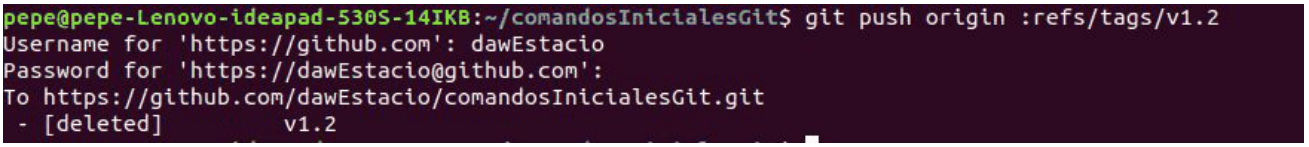
Eliminar etiquetes

De la mateixa manera que podem crear noves etiquetes, també podem eliminar-les. Per a això, simplement hem d'executar la instrucció:

```
git tag -d nom_tag
```

La instrucció anterior eliminarà la nostra etiqueta localment. Si volem que l'esborrat també s'execute en el servidor remot haurem d'executar:

```
git push origin :refs/tags/nom_tag
```



Etiquetatge tardà

També podem etiquetar commits molt de temps després d'haver-los fet. Suposem que el nostre historial llueix com el següent:

```
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git log --pretty=oneline
71b6ad05978b85daf310295d1c250fb108b3ab8a (HEAD -> master, tag: v1.0) Cambio conflictivo en la rama master
fafae5862261903095da6f1862842fc7dd7a60ae Cambio realizado en la rama provisional
a0b91a5dd96148fe448d10439b94c246d7c5c3e5 (origin/master) Realizado el merge de los cambios
d2fc53fe0a3906128a7eeed9cd7390908a530a22 Modificación del fichero por el primer usuario
6e6bc2364bf718f8e63ec1929048466ec2ef8bbf Modificación del primer fichero por el usuario 2
9ab1f121eff43518342c4d3d378819653f6fc167 Modificación del fichero por el usuario 2
c6d55c3896c6fb250796c12b08687bd68b88ef7f Creación de fichero por el usuario 2
eb9462aaef784a0952b0db4b5b750f1840225f54 Segunda modificación
c93df2609c3eac015a471389d299aa1d112cb2e5 Creado fichero primerFichero.txt
```

Ara, suposem que oblidem etiquetar el projecte en la seua versió v1.2, la qual correspon al commit "Segona modificació". Per etiquetar un commit passat, hem d'especificar el checksum del commit (o part d'ell) al final de l'ordre:

```
git tag -a v1.2 -m "Etiquetant versió 1.2" eb9462aaef784a
```

Si després d'executar la instrucció anterior, tornem a consultar el log:

```
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git tag -a v1.2 -m "Etiquetando versión 1.2" eb9462aae
f784a
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git log --pretty=oneline
71b6ad05978b85daf310295d1c250fb108b3ab8a (HEAD -> master, tag: v1.0) Cambio conflictivo en la rama master
fafae5862261903095da6f1862842fc7dd7a60ae Cambio realizado en la rama provisional
a0b91a5dd96148fe448d10439b94c246d7c5c3e5 (origin/master) Realizado el merge de los cambios
d2fc53fe0a3906128a7eeed9cd7390908a530a22 Modificación del fichero por el primer usuario
6e6bc2364bf718f8e63ec1929048466ec2ef8bbf Modificación del primer fichero por el usuario 2
9ab1f121eff43518342c4d3d378819653f6fc167 Modificación del fichero por el usuario 2
c6d55c3896c6fb250796c12b08687bd68b88ef7f Creación de fichero por el usuario 2
eb9462aaef784a0952b0db4b5b750f1840225f54 (tag: v1.2) Segunda modificación
c93df2609c3eac015a471389d299aa1d112cb2e5 Creado fichero primerFichero.txt
```

Compartir etiquetes

Per defecte, l'ordre git push no transfereix les etiquetes als servidors remots. Hem d'enviar les etiquetes de forma explícita al servidor després d'haver-les creades. Aquest procés és semblant al de compartir branques remotes: pots executar **git push origin [etiqueta]**.

```
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git push origin v1.2
Username for 'https://github.com': dawEstacio
Password for 'https://dawEstacio@github.com':
Contando objetos: 1, listo.
Escribiendo objetos: 100% (1/1), 176 bytes | 176.00 KiB/s, listo.
Total 1 (delta 0), reused 0 (delta 0)
To https://github.com/dawEstacio/comandosInicialesGit.git
 * [new tag]         v1.2 -> v1.2
```

Si volem enviar diverses etiquetes alhora, podem usar l'opció --tags de l'ordre git push. Això enviarà al servidor remot totes les etiquetes que encara no existeixen en ell.

```
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git push origin --tags
Username for 'https://github.com': dawEstacio
Password for 'https://dawEstacio@github.com':
Contando objetos: 7, listo.
Delta compression using up to 8 threads.
Comprimiendo objetos: 100% (7/7), listo.
Escribiendo objetos: 100% (7/7), 775 bytes | 387.00 KiB/s, listo.
Total 7 (delta 3), reused 0 (delta 0)
remote: Resolving deltas: 100% (3/3), completed with 1 local object.
To https://github.com/dawEstacio/comandosInicialesGit.git
 * [new tag]         v1.0 -> v1.0
```

Obtindre la versió del repositori etiquetada

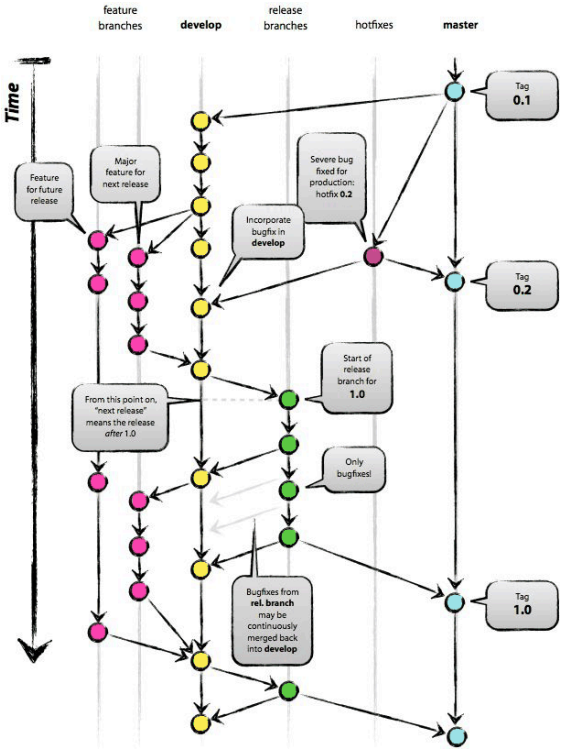
En Git, no pots obtindre (checkout) directament el codi del repositori associat a una etiqueta (no és una cosa que puguem moure). Si volem col·locar en el nostre directori de treball una versió del teu repositori que coincidisca amb alguna etiqueta, hem de crear una branca nova en aquesta etiqueta:

```
git checkout -b <nom_nova_branca> nom_tag
```

```
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git log --pretty=oneline
1e360b37f155ec0771f1ed9f6aef89f7749053f8 (HEAD -> master) Conflictos del merge resueltos
71b6ad05978b85daf310295d1c250fb108b3ab8a (tag: v1.0) Cambio conflictivo en la rama master
1a0d45a15c4249f5fc5a2ad5fa78d0822eafb5c6 (ramaProvisional) Cambio conflictivo en la rama provisional
fafae5862261903095da6f1862842fc7dd7a60ae Cambio realizado en la rama provisional
a0b91a5dd96148fe448d10439b94c246d7c5c3e5 (origin/master) Realizado el merge de los cambios
d2fc53fe0a3906128a7eeed9cd7390908a530a22 Modificación del fichero por el primer usuario
6e6bc2364bf718f8e63ec1929048466ec2ef8bbf Modificación del primer fichero por el usuario 2
9ab1f121eff43518342c4d3d378819653f6fc167 Modificación del fichero por el usuario 2
c6d55c3896c6fb250796c12b08687bd68b88ef7f Creación de fichero por el usuario 2
eb9462aaef784a0952b0db4b5b750f1840225f54 (tag: v1.2) Segunda modificación
c93df2609c3eac015a471389d299aa1d112cb2e5 Creado fichero primerFichero.txt
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git checkout -b rama_v1.2 v1.2
Cambiado a nueva rama 'rama_v1.2'
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git branch -a
  master
  ramaProvisional
* rama_v1.2
  remotes/origin/master
pepe@pepe-Lenovo-ideapad-530S-14IKB:~/comandosInicialesGit$ git log --pretty=oneline
eb9462aaef784a0952b0db4b5b750f1840225f54 (HEAD -> rama_v1.2, tag: v1.2) Segunda modificación
c93df2609c3eac015a471389d299aa1d112cb2e5 Creado fichero primerFichero.txt
```

5. GITFLOW

Gitflow és un **disseny de flux de treball de Git** que va ser publicat per primera vegada i popularitzat per Vincent Driessen en **nvie** i que **defineix un model estricte de ramificació** dissenyat al voltant de la publicació del projecte. És ideal per als projectes que tenen un cicle de publicació programat. Aquest flux de treball no afig cap concepte ni ordre nous més enllà del que es necessita per al flux de treball de branca de funció. En lloc d'això, assigna funcions molt específiques a les diferents branques i defineix com i quan han d'interactuar.



5.1 Per què implementar-lo?

- Augmenta la velocitat de lliurament de codi acabat a l'equip de proves.
- Disminueixen els errors humans en la barreja de les branques.
- Elimina la dependència de funcionalitats en el moment de lliurar codi per ser posat en producció.

5.2 Quan es recomana implementar-lo?

- L'equip de treball està format per més de dues (2) persones.
- S'empren metodologies àgils.
- El projecte té canvis freqüents i es requereix actualitzar l'ambient de producció garantint continuïtat en l'operació.
- El projecte té un nivell de complexitat considerable.
- Es desitja tindre un procés de suport a errors efectiu amb actualitzacions ràpides.

5.3 Funcionament

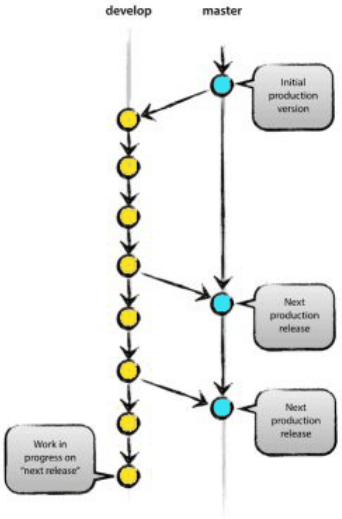
En lloc d'una única branca mestra, aquest flux de treball **utilitza dos tipus de branques** per registrar l'historial del projecte:

Les branques principals

En aquest esquema hi ha dues branques principals amb un temps de vida indefinit:

- master (origin/master): el codi apuntat per HEAD sempre conté un estat llest per a producció.
- develop (origin/develop): el codi apuntat per HEAD sempre conté els últims canvis desenvolupats per a la pròxima versió del programari. Quan el codi de la branca de

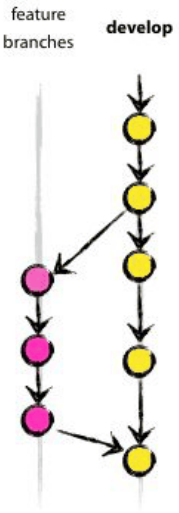
desenvolupament és prou estable, s'integra amb la branca master i una nova versió és llançada.



Les branques de funció

Per a tasques concretes, poden usar-se un altre tipus de branques, les quals tenen un temps de vida definit. És a dir, quan ja no són necessàries s'eliminen:

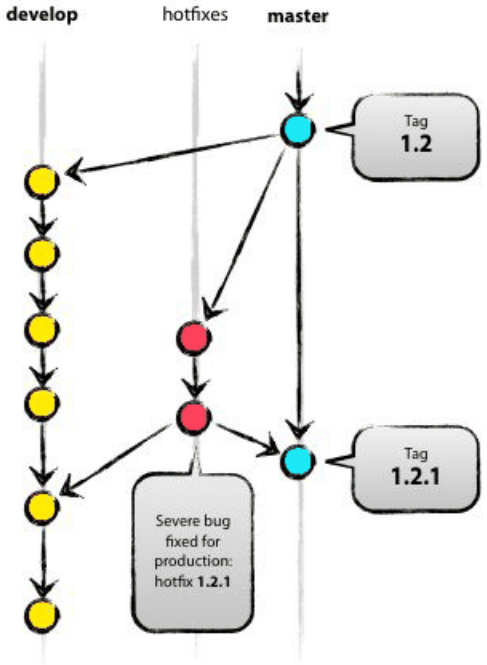
- Branques de funcionalitat (feature branches). Cada nova característica ha de residir en la seua pròpia branca, la qual pot ser guardada en el repositori remot per a resguard o col·laboració. Però, en lloc de ramificar des de master, **les branques feature usen develop com la seua branca pare. Quan una característica estiga completa, ha de ser fusionada amb develop**. Les característiques mai no haurien d'anar directament a la branca master.



- Branques de versió (release branches). **Una vegada la branca de desenvolupament haja adquirit prou característiques per a un llançament** (o una data de llançament està a prop), es crea una branca de versió des de la branca develop. En crear la branca s'inicia un nou cicle de llançament, la qual cosa significa que **en aquesta branca no haurien d'agregar-se noves característiques sinó a la branca develop; no obstant això**, si es troben **errors** en aquest

punt, aquests **haurien de ser corregits** i actualitzats en la branca de llançament tant com en la branca de desenvolupament.

- Branques de pedaç (hotfix branches). S'utilitzen per reparar ràpidament les publicacions de producció. És l'**única branca que hauria de bifurcar-se directament a partir de la mestra**. Una vegada que la solució estiga completa, hauria de fusionar-se en la mestra i la de desenvolupament (o la branca de publicació actual), i la mestra hauria d'etiquetar-se amb un número de versió actualitzat.



Branca de funció	Part de	Es fusiona amb	Convenció de noms
De funcionalitat (feature)	develop	develop	feature/nomFeature
De versió (release)	develop	develop i master	release-*
De pedaç (hotfix)	master	develop i master	hotfix-*

5.4 Extensió flow de Git (Git-Flow)

Git-Flow és un conjunt d'extensions per a Git que proveeixen ordres d'alt nivell per operar repositoris basats en el model de ramificacions de Vincent Driessen.

Instal·lació i configuració

Per poder començar a treballar amb l'extensió de Git Git-Flow, serà necessari instal·lar-la. Per a això, simplement hem d'executar la instrucció següent:

```
apt-get install git-flow
```

Una vegada instal·lat, si ens trobem en un projecte nou en el qual volem utilitzar git-flow, executarem:

```
git-flow-init

No branches exist yet. Base branches must be created now.
Branch name for production releases: [master]
Branch name for "next release" development: [develop]

How to name your supporting branch prefixes?
Feature branches? [feature/]
Release branches? [release/]
Hotfix branches? [hotfix/]
Support branches? [support/]
Version tag prefix? [] v
```

En executar l'última ordre, entrem en un menú interactiu que ens va preguntant les opcions que apareixen a dalt:

1. Ens pregunta el nom de la branca de producció i de la branca de desenvolupament (master i develop respectivament). Seleccionem els noms per defecte.
2. Després ens preguntarà els prefixos que assignarà a les branques de tipus feature, release, hotfix i support. Tornem a seleccionar els noms per defecte.
3. Finalment, ens pregunta el prefix que vulguem utilitzar per etiquetar les versions.

A partir d'ací, se'ns hauran creat les dues branques principals (master i develop) en el nostre repositori i podrem implementar noves releases, features i hotfix amb unes simples ordres sense necessitat de preocupar-nos per la creació i el merge de les branques necessàries per a això.

Principals instruccions de git-flow

Aquestes són les equivalències entre ordres de git-flow i la seua alternativa utilitzant simplement git:

gitflow	git
INICIALITZACIÓ	

`git flow init`	`git init`
	`git commit --allow-empty -m "Initial commit"`
	`git checkout -b develop master`

FEATURES	
`git flow feature start MYFEATURE`	`git checkout -b feature/MYFEATURE develop`

`git flow feature finish MYFEATURE`	`git checkout develop`
	`git merge --no-ff feature/MYFEATURE`
	`git branch -d feature/MYFEATURE`

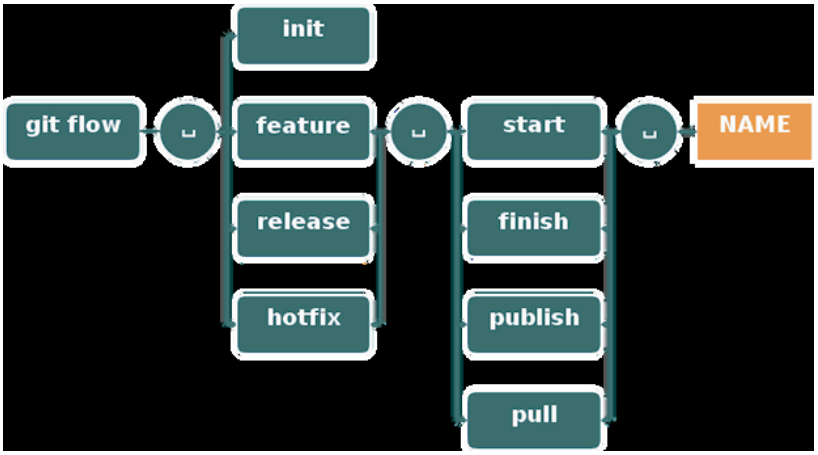
`git flow feature publish MYFEATURE`	`git checkout feature/MYFEATURE` `git push origin feature/MYFEATURE`
`git flow feature pull origin MYFEATURE`	`git checkout feature/MYFEATURE` `git pull --rebase origin feature/MYFEATURE`
RELEASES	
`git flow release start 1.2.0`	`git checkout -b release/1.2.0 develop`

`git flow release finish 1.2.0`	`git checkout master`
	`git merge --no-ff release/1.2.0`
	`git tag -a 1.2.0`
	`git checkout develop`
	`git merge --no-ff release/1.2.0`
	`git branch -d release/1.2.0`

`git flow release publish 1.2.0`	`git checkout release/1.2.0` `git push origin release/1.2.0`
HOTFIX	
`git flow hotfix start 1.2.1`	`git checkout -b hotfix/1.2.1`

`git flow hotfix finish 1.2.1`	`git checkout master`
	`git merge --no-ff hotfix/1.2.1`
	`git tag -a 1.2.1`
	`git checkout develop`
	`git merge --no-ff hotfix/1.2.1`
	`git branch -d hotfix/1.2.1`

`git flow hotfix publish 1.2.1`	`git checkout hotfix/1.2.1` `git push origin hotfix/1.2.1`



6. GITHUB PAGES

GitHub ens brinda eines molt útils per al treball en equip. Una d'aquestes és GitHub Pages, la qual permet allotjar llocs web estàtics sense necessitat de tindre coneixements en servidors. GitHub Pages permet dues modalitats de publicació:

- La primera és **"User site"** (només es podrà tindre un lloc d'aquest tipus per compte); en aquest cas el lloc web serà publicat en username.github.io (sent username el nom d'usuari del compte).
- La segona opció és **"Project site"** (projectes il·limitats), el qual serà publicat en username.github.io/repository (sent repository el nom del repositori).

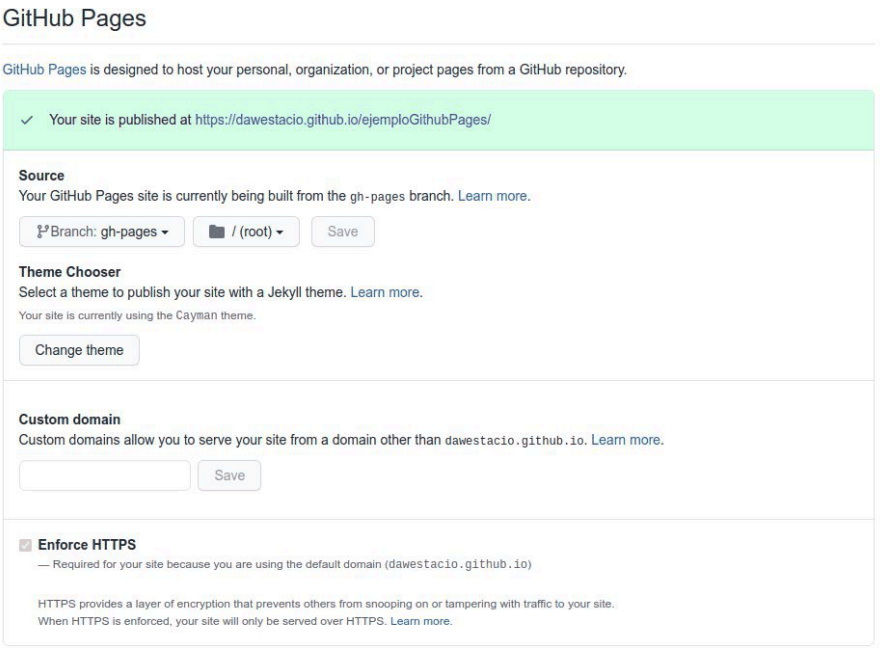
En tots dos casos, abans de començar hem de prendre en consideració el següent:

- Tindre un compte de GitHub creat.
- Tindre, com no, instal·lat git en el nostre equip.

Una vegada tinguem el nostre repositori, el pas següent consistirà a pujar els fitxers que vagen a ser utilitzats en el nostre site. Aquests poden ser tant fitxers html normals com fitxers en format .md (però no combinar tots dos tipus per evitar problemes). A més, aquests fitxers podran residir en la branca principal (master) o en una branca específica dedicada únicament a allotjar la documentació o site del nostre projecte (sol utilitzar-se en aquest cas una branca denominada gh-pages).

Després de crear el nostre repositori i disposar en ell de tots els fitxers del site pujats, haurem de procedir a habilitar GitHub Pages en el nostre magatzem de codi. Per a això, ens dirigirem a la secció de settings d'aquest i marcarem el check de la secció de GitHub Pages. A més:

- Seleccionarem la branca i el directori on es troben els fitxers del lloc.
- En cas d'optar per documentar el nostre codi en format .md, podem aplicar una plantilla jekyll per dotar d'un millor aspecte final el nostre site.



7. BONES PRÀCTIQUES EN GIT

Algunes bones pràctiques que es recomanen en utilitzar Git són:

- **Missatges de commit**
  Han de ser simples i clars. És aconsellable tindre en compte:
  - Separar l'assumpte del cos del missatge per una línia en blanc.
  - L'assumpte no ha de superar els 50 caràcters, ha d'iniciar-se amb lletra majúscula i no acabar amb punt. S'ha d'usar la forma imperativa en la seua redacció.
  - És aconsellable que el cos del missatge tinga línies amb una amplària de no més de 72 caràcters per facilitar-ne la lectura.
  - Utilitzar el cos del missatge per explicar què s'ha resolt i com.
- **Afegir només els fitxers amb els quals hem treballat**
  - Usar git add . no és la millor pràctica. El millor en aquests casos és afegir només els fitxers en els quals hàgem estat treballant; això evita que s'afigen molts fitxers "trash" que poden tindre una repercussió greu en la dimensió del repositori.
  - En el seu lloc, és convenient afegir els fitxers individualment (git add document.txt).
- **Evitar commits amb codis incomplets o barrejats**
  - Evitar, en la mesura que siga possible, pujar al repositori remot commits amb canvis de codi que suposen la resolució parcial d'un desenvolupament.
  - Realitzar commits per funcionalitat o requeriment. Això permet definir un abast en els commits. S'han d'incloure només fitxers que estiguen relacionats amb la funcionalitat que estiguem desenvolupant; això permet revertir (revert) confirmacions en un futur sense cap problema.

8. BIBLIOGRAFIA

Documentació oficial de Git - enllaç
Apunts IOC - enllaç
Pàgina oficial d'Atlassian - enllaç
Git flow cheatsheet - enllaç